

Análise de Melhorias — Kalles PDV

Insights de Spring, JPA e Java para estudo e possível implementação. Cada seção explica **o que é, por que é útil no contexto de PDV**, e **como ficaria** no projeto.

Índice

1. Bean Validation nos DTOs do Core
 2. GlobalExceptionHandler unificado
 3. JPA Auditing (@CreatedDate, @LastModifiedDate)
 4. Optimistic Locking (@Version)
 5. N+1 Problem e @EntityGraph
 6. Column Constraints e Indexes
 7. Precisão de BigDecimal (precision, scale)
 8. Spring Profiles e Configuração por Ambiente
 9. Database Migrations (Flyway/Liquibase)
 10. Spring Events para Desacoplamento
 11. OpenAPI/Swagger no SaleController
 12. Padronização de Idioma nas Mensagens
 13. Exceções Específicas no CheckoutSessionService
 14. @PrePersist / @PreUpdate Callbacks
 15. Lombok @Builder para objetos complexos
 16. Hibernate format_sql para Debug
 17. Physical Naming Strategy
 18. Tabela Resumo — Prioridades
-

1. Bean Validation nos DTOs do Core

O que é

Jakarta Bean Validation (`jakarta.validation`) permite declarar regras de validação diretamente nos campos dos DTOs usando annotations como `@NotNull`, `@NotBlank`, `@Positive`, `@DecimalMin`, etc. O Spring as aplica automaticamente quando o parâmetro do controller é anotado com `@Valid`.

Situação atual

O `OpenSessionRequest` do cashregister **já usa** (`@NotBlank`, `@NotNull`, `@DecimalMin`), mas os DTOs do core (`AddItemRequest`, `PaymentRequest`) **não têm nenhuma validação**, e o `SaleController` **não usa** `@Valid` em nenhum `@RequestBody`.

Por que é útil no PDV

Sem validação, um request com `amount: null` ou `amount: -10` chega até o service, onde o `if (amount == null)` manual faz a checagem. Com Bean Validation, o Spring rejeita o request na camada de controller com HTTP 400 **antes** de chegar ao service — fail-fast + menos código manual.

Como ficaria

```
// AddItemRequest.java
public record AddItemRequest(
    @NotNull(message = "Tipo de código é obrigatório")
    ProductCodeType type,

    @NotBlank(message = "Código do produto é obrigatório")
    String code
) {}

// PaymentRequest.java
public record PaymentRequest(
    @NotNull(message = "Método de pagamento é obrigatório")
    PaymentMethod method,

    @NotNull(message = "Valor do pagamento é obrigatório")
    @Positive(message = "Valor do pagamento deve ser positivo")
    BigDecimal amount
) {}

// SaleController.java – adicionar @Valid nos @RequestBody
@PostMapping("/{sessionToken}/items")
public ResponseEntity<SaleResponse> addItem(
    @PathVariable String sessionToken,
    @Valid @RequestBody AddItemRequest request) { ... }

@PostMapping("/{sessionToken}/payments")
public ResponseEntity<SaleResponse> addPayment(
    @PathVariable String sessionToken,
    @Valid @RequestBody PaymentRequest request) { ... }
```

O que estudar

- Pesquise: `jakarta.validation.constraints` — lista completa de annotations
- Pesquise: `@Validated` vs `@Valid` — diferenças (groups)
- Pesquise: `MethodArgumentNotValidException` — já tratada no `GlobalExceptionHandler`

2. GlobalExceptionHandler unificado

O que é

`@RestControllerAdvice` é um `@ControllerAdvice` + `@ResponseBody` que intercepta exceções lançadas por **todos** os controllers e retorna respostas HTTP padronizadas. `ProblemDetail` (RFC 7807) é o formato padrão do Spring para respostas de erro.

Situação atual

Existe um handler em `cashregister.controller.GlobalExceptionHandler`, mas ele só trata exceções do `cashregister` (`ActiveSessionAlreadyExistsException`, `CashRegisterNotFoundException`, `OperatorNotFoundException`). As exceções do `core` (`NotFoundException`, `ForbiddenOperationException`, `IllegalStateException` das transições de estado) **não são tratadas** — o Spring retorna um stack trace genérico com HTTP 500.

Além disso, o handler existente está no pacote `cashregister.controller`, mas existe um arquivo vazio em `api.handler.GlobalExceptionHandler`.

Como ficaria

```
// api/handler/GlobalExceptionHandler.java – handler unificado
@RestControllerAdvice
public class GlobalExceptionHandler {

    // Exceções do core
    @ExceptionHandler(NotFoundException.class)
    public ProblemDetail handleNotFound(NotFoundException ex) {
        ProblemDetail problem = ProblemDetail.forStatusAndDetail(
            HttpStatus.NOT_FOUND, ex.getMessage());
        problem.setTitle("Recurso não encontrado");
        return problem;
    }

    @ExceptionHandler(ForbiddenOperationException.class)
    public ProblemDetail handleForbidden(ForbiddenOperationException ex) {
        ProblemDetail problem = ProblemDetail.forStatusAndDetail(
            HttpStatus.FORBIDDEN, ex.getMessage());
        problem.setTitle("Operação não permitida");
        return problem;
    }

    @ExceptionHandler(IllegalStateException.class)
    public ProblemDetail handleIllegalState(IllegalStateException ex) {
        ProblemDetail problem = ProblemDetail.forStatusAndDetail(
            HttpStatus.CONFLICT, ex.getMessage());
        problem.setTitle("Operação inválida para o estado atual");
        return problem;
    }

    @ExceptionHandler(IllegalArgumentException.class)
    public ProblemDetail handleIllegalArgument(IllegalArgumentException ex) {
        ProblemDetail problem = ProblemDetail.forStatusAndDetail(
            HttpStatus.BAD_REQUEST, ex.getMessage());
        problem.setTitle("Argumento inválido");
        return problem;
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ProblemDetail handleValidation(MethodArgumentNotValidException ex) {
        String errors = ex.getBindingResult().getFieldErrors().stream()
            .map(e -> e.getField() + ": " + e.getDefaultMessage())
```

```

        .collect(Collectors.joining(", "));
        ProblemDetail problem = ProblemDetail.forStatusAndDetail(
            HttpStatus.BAD_REQUEST, errors);
        problem.setTitle("Erro de validação");
        return problem;
    }

    // Exceções do cashregister
    @ExceptionHandler(ActiveSessionAlreadyExistsException.class)
    public ProblemDetail handleActiveSession(ActiveSessionAlreadyExistsException
ex) {
        ProblemDetail problem = ProblemDetail.forStatusAndDetail(
            HttpStatus.CONFLICT, ex.getMessage());
        problem.setTitle("Sessão ativa já existe");
        return problem;
    }

    @ExceptionHandler({CashRegisterNotFoundException.class,
OperatorNotFoundException.class})
    public ProblemDetail handleCashRegisterNotFound(RuntimeException ex) {
        ProblemDetail problem = ProblemDetail.forStatusAndDetail(
            HttpStatus.NOT_FOUND, ex.getMessage());
        problem.setTitle("Recurso não encontrado");
        return problem;
    }
}

```

O que estudar

- Pesquise: `ProblemDetail` (RFC 7807) — formato padrão de erro REST
- Pesquise: `@RestControllerAdvice(basePackages = ...)` — scoped handlers
- Pesquise: `@Order` / `@Priority` — precedência entre múltiplos handlers

3. JPA Auditing (@CreatedDate, @LastModifiedDate)

O que é

Spring Data JPA Auditing preenche automaticamente campos de data (criação, última modificação) e até quem criou/modificou (`@CreatedBy`). Elimina o `LocalDateTime.now()` manual espalhado nas entities.

Situação atual

- `SaleCancellation.canceledAt = LocalDateTime.now()` — inicializado na declaração do campo
- `ItemRemovedSale.timestamp = LocalDateTime.now()` — idem
- `Payment.createdAt` — inicializado no construtor com `LocalDateTime.now()`

Funciona, **mas** tem um problema sutil: se você criar o objeto e persistir 5 segundos depois, o timestamp reflete a criação do objeto Java, não a persistência no banco.

Como ficaria

```
// 1. Habilitar na Application
@SpringBootApplication
@EnableJpaAuditing // ← adicionar
public class KallesSaleApplication { ... }

// 2. Criar uma classe base (opcional, mas DRY)
@MappedSuperclass
@EntityListeners(AuditingEntityListener.class)
@Getter
public abstract class BaseAuditableEntity {

    @CreatedDate
    @Column(updatable = false)
    private LocalDateTime createdAt;

    @LastModifiedDate
    private LocalDateTime updatedAt;
}

// 3. Usar nas entities – exemplo Payment
@Entity
public class Payment extends BaseAuditableEntity {
    // remove o campo createdAt manual
    // remove LocalDateTime.now() do construtor
}

// 4. Ou individualmente sem herança
@Entity
@EntityListeners(AuditingEntityListener.class)
public class SaleCancellation {
    @CreatedDate
    @Column(updatable = false)
    private LocalDateTime canceledAt;
    // remove o = LocalDateTime.now()
}
```

O que estudar

- Pesquise: `@EnableJpaAuditing` — ativação
- Pesquise: `AuditingEntityListener` — como funciona por baixo
- Pesquise: `@CreatedBy` + `AuditorAware<T>` — para capturar o operador logado automaticamente (útil futuro)
- Pesquise: `@MappedSuperclass` — herança JPA sem tabela separada

4. Optimistic Locking (`@Version`)

O que é

`@Version` adiciona um campo de versão na entity. Toda vez que o Hibernate faz UPDATE, ele inclui `WHERE version = X` na query. Se duas transações tentarem atualizar a mesma row simultaneamente, a segunda recebe `OptimisticLockException`.

Por que é crítico no PDV

Imagine dois operadores no mesmo caixa (troca de turno, ou mesmo terminal acessado por duas abas). Sem `@Version`:

1. Operador A lê a venda (total = R\$50)
2. Operador B lê a mesma venda (total = R\$50)
3. Operador A adiciona item → total = R\$80, salva
4. Operador B adiciona pagamento de R\$50 → pensa que total é R\$50, salva → **sobrescreve** o item do operador A

Com `@Version`, o passo 4 falha com exceção, e o sistema pode re-ler e tentar novamente.

Como ficaria

```
@Entity
public class Sale {
    @Id @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;

    @Version    // ← só isso
    private Long version;

    // ... resto igual
}
```

O Spring/Hibernate cuida do resto. Você só precisa tratar `OptimisticLockException` no `GlobalExceptionHandler`:

```
@ExceptionHandler(OptimisticLockException.class)
public ProblemDetail handleOptimisticLock(OptimisticLockException ex) {
    ProblemDetail problem = ProblemDetail.forStatusAndDetail(
        HttpStatus.CONFLICT,
        "A venda foi modificada por outra operação. Tente novamente.");
    problem.setTitle("Conflito de concorrência");
    return problem;
}
```

O que estudar

- Pesquise: `@Version` JPA — Optimistic Locking
- Pesquise: `OptimisticLockException` vs `StaleObjectStateException` (Hibernate)

- Pesquisa: Pessimistic Locking (`@Lock(LockModeType.PESSIMISTIC_WRITE)`) — alternativa para cenários de alta contenção

5. N+1 Problem e `@EntityGraph`

O que é

O **N+1 problem** é o bug de performance mais comum em JPA. Acontece quando você carrega uma entity e depois acessa suas coleções lazy, gerando 1 query para a entity + N queries para cada coleção.

Onde acontece no projeto

`Sale` tem 4 `@OneToMany` (items, payments, itemsRemoved, cancellations) — todos **LAZY** por padrão. Quando o controller faz `SaleResponse.from(sale)`, ele acessa **todas** as 4 listas:

```
SELECT * FROM sale WHERE ...;           -- 1 query
SELECT * FROM sale_item WHERE sale_id = ?; -- +1
SELECT * FROM payment WHERE sale_id = ?;   -- +1
SELECT * FROM item_removed_sale WHERE sale_id = ?; -- +1
SELECT * FROM sale_cancellation WHERE sale_id = ?; -- +1
= 5 queries para UMA venda
```

Como resolver

Opção A: `@EntityGraph` (declarativa)

```
public interface SaleRepository extends JpaRepository<Sale, UUID> {

    @EntityGraph(attributePaths = {"items", "items.product", "payments",
                                   "itemsRemoved", "cancellations"})
    @Query("SELECT s FROM Sale s WHERE s.sessionToken = :token AND s.state IN :states")
    Optional<Sale> findBySessionTokenAndStateIn(
        @Param("token") String token, @Param("states") List<String> states);
}
```

Opção B: `JOIN FETCH` na JPQL

```
@Query("""
    SELECT s FROM Sale s
    LEFT JOIN FETCH s.items i
    LEFT JOIN FETCH i.product
    LEFT JOIN FETCH s.payments
    WHERE s.sessionToken = :token AND s.state IN :states
    """)
Optional<Sale> findBySessionTokenAndStateIn(...);
```

⚠ **Cuidado:** JOIN FETCH de múltiplos `@OneToMany` pode gerar Cartesian Product. O Hibernate 6+ usa `@Fetch(FetchMode.SUBSELECT)` ou múltiplas queries como alternativa. Estude `MultipleBagFetchException`.

O que estudar

- Pesquise: `@EntityGraph` Spring Data JPA
- Pesquise: JOIN FETCH JPQL — e quando usar LEFT vs INNER
- Pesquise: `MultipleBagFetchException` — e por que trocar `List` por `Set` resolve
- Pesquise: `spring.jpa.open-in-view=false` — e por que é recomendado desabilitar
- Pesquise: `hibernate.default_batch_fetch_size` — alternativa simples (config only)

6. Column Constraints e Indexes

O que é

`@Column` define constraints (nullable, unique, length) que o Hibernate usa para gerar DDL. `@Table(indexes = ...)` cria índices no banco. Isso afeta performance e integridade de dados.

Situação atual

As entities do core (`Sale`, `SaleItem`, `Payment`, `Product`) **não definem** constraints nas colunas. Já as entities do cashregister definem (`@Column(nullable = false, unique = true)`).

Como ficaria

```
@Entity
@Table(indexes = {
    @Index(name = "idx_sale_session_token", columnList = "sessionToken"),
    @Index(name = "idx_sale_state", columnList = "state")
})
public class Sale {
    @Column(nullable = false)
    private String sessionToken;
    // ...
}

@Entity
@Table(indexes = {
    @Index(name = "idx_product_internal_code", columnList = "internalCode", unique
= true),
    @Index(name = "idx_product_barcode", columnList = "barcode", unique = true)
})
public class Product {
    @Column(nullable = false, length = 150)
    private String name;

    @Column(nullable = false, unique = true, length = 20)
    private String internalCode;
```



```
@Column(unique = true, length = 50)
private String barcode;

@Column(nullable = false, precision = 19, scale = 2)
private BigDecimal price;

@Column(nullable = false)
private boolean active;
}
```

O que estudar

- Pesquise: `@Column` attributes — `nullable`, `unique`, `length`, `precision`, `scale`, `columnDefinition`
- Pesquise: `@Table(indexes = ...)` e `@Index`
- Pesquise: Database indexing fundamentals — B-tree, quando criar, impacto em INSERT

7. Precisão de BigDecimal (`precision`, `scale`)

O que é

Sem `@Column(precision = ..., scale = ...)`, o Hibernate cria colunas `numeric(19,2)` ou o padrão do dialeto PostgreSQL. Para valores monetários no Brasil, é importante garantir `scale = 2` (centavos) e `precision` adequada.

Como ficaria

```
// Em todas as entities com BigDecimal monetário
@Column(nullable = false, precision = 19, scale = 2)
private BigDecimal amount;

@Column(nullable = false, precision = 19, scale = 2)
private BigDecimal changeAmount = BigDecimal.ZERO;

@Column(precision = 19, scale = 2)
private BigDecimal subtotal = BigDecimal.ZERO;

@Column(precision = 19, scale = 2)
private BigDecimal total = BigDecimal.ZERO;

@Column(precision = 19, scale = 2)
private BigDecimal amountDue = BigDecimal.ZERO;
```

O que estudar

- Pesquise: `BigDecimal` e `RoundingMode` — `setScale(2, RoundingMode.HALF_UP)` para evitar decimais infinitos
- Pesquise: Por que **nunca** usar `double/float` para dinheiro

8. Spring Profiles e Configuração por Ambiente

O que é

Spring Profiles permitem ter configurações diferentes para dev, test e produção. Sem profiles, o `application.yml` vale para **todos** os ambientes.

Situação atual

O `application.yml` hardcodea credenciais do PostgreSQL. Os testes não têm configuração específica (por isso `KallesSaleApplicationTests` falha — tenta conectar no PostgreSQL quando deveria usar H2).

Como ficaria

```
# application.yml – configuração base
spring:
  application:
    name: kalles-sale
  jpa:
    hibernate:
      ddl-auto: validate # produção = validate ou none
    properties:
      hibernate:
        format_sql: true

---
# application-dev.yml
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/kalles
    username: kalles_admin
    password: kalles_password
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true

---
# application-test.yml (src/test/resources/)
spring:
  datasource:
    url: jdbc:h2:mem:testdb;MODE=PostgreSQL
    driver-class-name: org.h2.Driver
    username: sa
    password:
  jpa:
    hibernate:
      ddl-auto: create-drop
    show-sql: false
```

Ativar: `SPRING_PROFILES_ACTIVE=dev` no ambiente ou na IDE.

O que estudar

- Pesquise: `@Profile("test")` — para beans específicos de teste
- Pesquise: `@ActiveProfiles("test")` — para testes de integração
- Pesquise: `@TestPropertySource` — override de propriedades em testes
- Pesquise: `spring.profiles.active` vs `spring.profiles.default`

9. Database Migrations (Flyway/Liquibase)

O que é

Em vez de `ddl-auto: update` (que **nunca** deve ser usado em produção), ferramentas de migration versionam o schema do banco com scripts SQL (`V1__create_sale.sql`, `V2__add_payment.sql`).

Por que importa

`ddl-auto: update` **nunca** remove colunas/tabelas, pode perder dados, e não tem rollback. Em produção real, é um desastre esperando para acontecer.

Como ficaria (Flyway)

```
<!-- kalles-sale/pom.xml -->
<dependency>
  <groupId>org.flywaydb</groupId>
  <artifactId>flyway-core</artifactId>
</dependency>
<dependency>
  <groupId>org.flywaydb</groupId>
  <artifactId>flyway-database-postgresql</artifactId>
</dependency>
```

```
src/main/resources/db/migration/
├─ V1__create_operators.sql
├─ V2__create_products.sql
├─ V3__create_sales.sql
├─ V4__create_sale_items.sql
├─ V5__create_payments.sql
└─ V6__create_cancellations.sql
```

```
# application.yml
spring:
  jpa:
    hibernate:
      ddl-auto: validate # apenas valida que o schema bate com as entities
```

```
flyway:
  enabled: true
  locations: classpath:db/migration
```

O que estudar

- Pesquise: Flyway — `V{version}__{description}.sql` naming convention
- Pesquise: Flyway vs Liquibase — trade-offs
- Pesquise: `spring.flyway.baseline-on-migrate=true` — para projetos existentes

10. Spring Events para Desacoplamento

O que é

`ApplicationEventPublisher` permite publicar eventos de domínio que são consumidos por listeners (`@EventListener` / `@TransactionalEventListener`). Isso desacopla o "o que aconteceu" do "o que fazer quando aconteceu".

Situação atual

`Sale.doCancel()` cria diretamente um `SaleCancellation` e adiciona na lista. `Sale.doRemoveItem()` cria um `ItemRemovedSale` e adiciona. A entidade está fazendo **domínio + auditoria** no mesmo lugar.

Como ficaria

```
// Evento de domínio
public record SaleCancelledEvent(UUID saleId, UUID requestedById, UUID
authorizedById) {}

// No service, após cancelar
@Transactional
public Sale cancelSale(String sessionToken, UUID operatorId) {
    // ... busca sale, operador, cancela ...
    sale.cancel(operator, null);
    Sale saved = saleRepository.save(sale);

    eventPublisher.publishEvent(new SaleCancelledEvent(
        saved.getId(), operatorId, null));

    return saved;
}

// Listener separado
@Component
public class SaleAuditListener {

    @TransactionalEventListener(phase = TransactionPhase.BEFORE_COMMIT)
    public void onSaleCancelled(SaleCancelledEvent event) {
        // cria SaleCancellation, salva log, notifica, etc.
    }
}
```

```
}  
}
```

Quando vale a pena

Para o projeto atual, pode ser **over-engineering**. Mas é um pattern importante quando:

- Múltiplos side-effects (email, log, notificação, atualizar estoque)
- Módulos distintos precisam reagir ao mesmo evento

O que estudar

- Pesquise: `ApplicationEventPublisher` — publicar eventos
- Pesquise: `@EventListener` vs `@TransactionalEventListener` — diferença de timing
- Pesquise: `TransactionPhase.AFTER_COMMIT` — para ações que não devem executar se a transação falhar
- Pesquise: `AbstractAggregateRoot` do Spring Data — `registerEvent()` diretamente na entity

11. OpenAPI/Swagger no SaleController

Situação atual

O `CashRegisterSessionController` já usa `@Tag` e `@Operation` do springdoc, mas o `SaleController` não tem nenhuma annotation de documentação.

Como ficaria

```
@RestController  
@RequestMapping("/api/sales")  
@RequiredArgsConstructor  
@Tag(name = "Vendas", description = "Operações de venda do PDV")  
public class SaleController {  
  
    @PostMapping("/{sessionToken}/items")  
    @Operation(summary = "Adicionar item à venda",  
              description = "Adiciona um produto pelo código interno ou código de barras")  
    public ResponseEntity<SaleResponse> addItem(...) { ... }  
  
    @PostMapping("/{sessionToken}/payments")  
    @Operation(summary = "Registrar pagamento",  
              description = "Processa pagamento com método especificado (CASH, PIX, etc.)")  
    public ResponseEntity<SaleResponse> addPayment(...) { ... }  
  
    @PostMapping("/{sessionToken}/complete")  
    @Operation(summary = "Finalizar venda",  
              description = "Marca a venda como COMPLETED após pagamento total")
```

```
public ResponseEntity<SaleResponse> completeSale(...) { ... }  
}
```

A UI fica disponível em <http://localhost:8080/swagger-ui.html> — a dependência springdoc já está no pom.xml.

O que estudar

- Pesquise: [springdoc-openapi](#) — configuration properties
- Pesquise: [@Schema](#) — para documentar DTOs e campos
- Pesquise: [@ApiResponse](#) — documentar códigos de retorno e erros

12. Padronização de Idioma nas Mensagens

Situação atual

Mensagens misturadas:

- PT-BR: "Produto não encontrado com o código interno: " (SaleService)
- EN: "No active sale found for this session." (PaymentService)
- EN: "Cannot process payment: sale has no items." (PaymentService)
- PT-BR: "Não é possível adicionar itens no estado: " (AbstractSaleState)

Recomendação

Padronizar em **PT-BR** (público do PDV são operadores brasileiros) ou usar [MessageSource](#) do Spring para internacionalização (i18n) se futuro multilíngue for possível.

O que estudar

- Pesquise: [MessageSource](#) e [messages.properties](#) — i18n no Spring
- Pesquise: [LocaleResolver](#) — para resolver idioma por request

13. Exceções Específicas no CheckoutSessionService

Situação atual

```
// CheckoutSessionService.java  
throw new RuntimeException("Sessão de caixa não encontrada: " + sessionToken);  
throw new RuntimeException("Sessão de caixa não está aberta: " + sessionToken);
```

[RuntimeException](#) genérica. O [GlobalExceptionHandler](#) não consegue mapear isso para um HTTP status específico.

Como ficaria

```
// Usar NotFoundException para sessão não encontrada
Session session = findByToken(sessionToken)
    .orElseThrow(() -> new NotFoundException(
        "Sessão de caixa não encontrada: " + sessionToken));

// Criar uma exceção específica para sessão fechada
if (!session.isOpen())
    throw new IllegalStateException(
        "Sessão de caixa não está aberta: " + sessionToken);
```

14. @PrePersist / @PreUpdate Callbacks

O que é

JPA Entity Lifecycle Callbacks executam lógica automaticamente antes de persist/update. Alternativa ao auditing para lógica customizada.

Quando usar em vez de @CreatedDate

Quando você precisa de lógica mais complexa (ex: recalcular totais antes de salvar).

```
@Entity
public class Payment {

    private LocalDateTime createdAt;

    @PrePersist
    protected void onCreate() {
        this.createdAt = LocalDateTime.now();
    }
}
```

O que estudar

- Pesquise: @PrePersist, @PostPersist, @PreUpdate, @PostUpdate, @PreRemove
- Pesquise: @EntityListeners — para extrair callbacks em classe separada

15. Lombok @Builder para objetos complexos

O que é

@Builder gera um builder pattern fluente. Útil para objetos com muitos campos opcionais.

Onde seria útil

```
// Em vez de:
Payment payment = new Payment(sale, method, amount, changeAmount,
                              result.transactionId(), result.confirmed());

// Com @Builder:
Payment payment = Payment.builder()
    .sale(sale)
    .method(method)
    .amount(amount)
    .changeAmount(changeAmount)
    .transactionId(result.transactionId())
    .confirmed(result.confirmed())
    .build();
```

⚠ `@Builder` + `@NoArgsConstructor` + `@AllArgsConstructor` do JPA requer `@Builder(toBuilder = true)` ou `@Builder` na classe + `@AllArgsConstructor(access = AccessLevel.PACKAGE)`.
Pesquise as combinações válidas.

O que estudar

- Pesquisa: `@Builder` Lombok + JPA — armadilhas comuns
- Pesquisa: `@Builder.Default` — para valores default (`BigDecimal.ZERO`)

16. Hibernate `format_sql` para Debug

Situação atual

`show-sql: true` está ativo, mas sem formatação. O SQL sai numa linha só, ilegível.

Como ficaria

```
spring:
  jpa:
    show-sql: true
    properties:
      hibernate:
        format_sql: true # formata o SQL com indentação
        highlight_sql: true # se o terminal suportar, colore o SQL
        use_sql_comments: true # adiciona comentário com o nome da entity/query
```

O que estudar

- Pesquisa: `p6spy` — logging de query com bind parameters (show-sql não mostra os `?` preenchidos)

17. Physical Naming Strategy

O que é

Define como nomes Java são mapeados para nomes SQL. Spring Boot usa `SpringPhysicalNamingStrategy` por padrão: `SaleItem` → `sale_item`, `sessionToken` → `session_token`.

Situação atual

Funciona automaticamente, mas não está explicitamente documentado. Entities do core não usam `@Table` nem `@Column(name = ...)`, confiando na strategy automática.

Entities do cashregister **definem** `@Table(name = "cash_register_sessions")` — inconsistência com o core.

Recomendação

Manter a strategy padrão do Spring, mas padronizar: ou todas definem `@Table`, ou nenhuma (confia na strategy). Para PDV, a strategy padrão é suficiente.

O que estudar

- Pesquise: `ImplicitNamingStrategy` vs `PhysicalNamingStrategy`
- Pesquise: `spring.jpa.hibernate.naming.physical-strategy`

18. Tabela Resumo — Prioridades

#	Melhoria	Impacto	Esforço	Prioridade
1	Bean Validation nos DTOs	Segurança + DX	Baixo	Alta
2	GlobalExceptionHandler unificado	UX da API	Baixo	Alta
4	Optimistic Locking (<code>@Version</code>)	Integridade	Muito baixo	Alta
6	Column Constraints + Indexes	Performance + Integridade	Médio	Alta
7	Precisão BigDecimal	Integridade financeira	Baixo	Alta
8	Spring Profiles	Operação	Médio	Média
9	Database Migrations (Flyway)	Operação	Médio	Média
12	Padronizar idioma mensagens	Manutenção	Baixo	Média
13	Exceções específicas no Session	Manutenção	Baixo	Média
3	JPA Auditing	DRY	Baixo	Baixa
5	N+1 / EntityGraph	Performance	Médio	Baixa*
11	Swagger no SaleController	Documentação	Baixo	Baixa
14	@PrePersist callbacks	DRY	Baixo	Baixa
15	Lombok @Builder	Legibilidade	Baixo	Baixa

#	Melhoria	Impacto	Esforço	Prioridade
16	format_sql	Debug	Muito baixo	☒ Baixa
10	Spring Events	Arquitetura	Alto	☐ Futuro
17	Naming Strategy docs	Manutenção	Muito baixo	☐ Futuro

* N+1 é prioridade baixa agora porque PDV tem poucos itens por venda. Em escala, sobe para alta.

Como Estudar

Para cada item, a sequência recomendada:

1. **Documentação oficial** — [Spring Data JPA Reference](#), [Jakarta Persistence Spec](#)
2. **Testar no projeto** — criar uma branch de experimento e implementar
3. **Verificar os testes** — confirmar que os 95 testes continuam passando
4. **Commit incremental** — uma melhoria por commit